

Projeto 2 – Aplicação Back-end REST API

Objetivos

O objetivo deste projeto é desenvolver uma REST API totalmente funcional de suporte à aplicação Front-end desenvolvido no Projeto 1. A API permitirá aos utilizadores realizar operações CRUD (Create, Read, Update, Delete) em diferentes *endpoints* e será desenvolvida em Python com o recurso à *framework* Flask.

Para armazenamento e manipulação de dados, será utilizada a base de dados MongoDB, com uso de Queries aos documentos.

Project Setup

1. Criar diretoria do projeto. Neste exemplo vamos chamar 'backend'. Todos os ficheiros do projeto vão estar localizados nesta pasta:

```
> mkdir backend  
> cd backend
```

2. Criar e ativar *virtual environment* (permite instalar dependências específicas ao projeto). Se não tiver o virtual environment instalado: `pip install virtualenv`

```
backend> python -m venv venv  
backend> .\venv\Scripts\activate
```

3. Instalar Flask

```
(venv) backend> python -m pip install Flask
```

4. Instalar Pymongo. Biblioteca que permite conectar e interagir com a base de dados MongoDB, facilitando as operações sobre documentos diretamente na aplicação Python.

```
(venv) backend> pip install pymongo
```

5. Instalar CORS. “Cross-Origin Resource Sharing” - mecanismo de segurança para impedir *HTTP requests* por domínios diferentes.

```
(venv) backend> pip install -U flask-cors
```

Files Setup

Criar ficheiro app.py e copiar o código abaixo.

```
from flask import Flask
from pymongo import MongoClient
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

client = MongoClient('MONGODB_CONECTION_STRING',
                    tls=True,
                    tlsAllowInvalidCertificates=True)
db = client["DATABASE_NAME"]

@app.route("/products", methods=["GET"])
def get_products():
    # GET todos os produtos
    return (...)
```

1. Alterar MONGODB_CONECTION_STRING: com o URL da vossa base de dados MongoDB;
2. Alterar DATABASE_NAME: com o nome da base de dados;

Run App:

```
flask --app app run --debug
```

Funcionalidades

#	Route	Endpoint	Descrição
1	GET	/api/v1/products	Lista de produtos com paginação. (ex: ?page=1&limit=10)
2	GET	/api/v1/products/<int:id>	Retorna produto pelo campo <i>_id</i>
3	POST	/api/v1/products	Adicionar produtos ou múltiplos produtos { <i>authentication required</i> }
4	DELETE	/api/v1/products/<int:id>	Remover produto pelo campo <i>_id</i> { <i>authentication required</i> }
5	PUT	/api/v1/products/<int:id>	Update produto { <i>authentication required</i> }
6	GET	/api/v1/products/total	Número total de produtos
7	GET	/api/v1/products/categorias/<categorias>	Retorna lista de produtos por categoria com paginação
8	GET	/api/v1/products/price	Lista de produtos entre um valor mínimo e um valor máximo, ordenado por ordem crescente ou decrescente. Paginação
9	POST	/api/v1/user/signup	Regista novo <i>user</i> na base de dados { <i>username: ""</i> , <i>password: ""</i> , <i>confirmed: False</i> }
10	POST	/api/v1/user/login	Autêntica o <i>user</i> {confirma que o user existe na base de dados, e se tiver o campo <i>confirmed</i> igual a True cria e envia JWT token}
11	POST	/api/v1/user/confirmation	Admin user confirma registo de novos utilizadores. { <i>authentication required</i> }

Avaliação

- Qualidade e organização do código do projeto;
- Correta definição de *routes*;

- Qualidade da solução de programação implementada;
- Correta definição das Queries MongoDB;
- Cumprimentos de todas as funcionalidades;
- Correta definição de funções;
- Correto tratamento de erros, com especificação de código de erro;
- Discussão.

O que têm de entregar

Até dia 4 de Junho devem entregar, ficheiro zip com:

- Pasta do projeto ou GitHub link;

Nota: Data de Apresentação e discussão individual 11 de Junho.

Qualquer dúvida que tenham, contactem o docente.

Bom trabalho.